

Document number	P3812R0
Date	2025-08-16
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	Core Working Group Evolution Working Group

const and & in default member functions

Table of contents

- const and & in default member functions
 - Abstract
 - Motivation
 - Impact on the standard
 - References

Abstract

Allow compilers to be able to generate the default copy assignment operator for classes that have `const` and `&` members by making the class `transparently replaceable` if it has a default copy constructor. Allow compilers to be able to generate the default move assignment operator for classes that have `const` and `&` members by making the class `transparently replaceable` if it has a default move constructor. See 6.7.4 Lifetime [basic.life] ¶ 9. ^[1]

C++26 style	This paper (proposed)

```

// least privilege
class leastp final
{
public:
    leastp(std::vector<int>& v, int i)
        : v{v}, i{i} {}

    leastp(const leastp&) = default;
    leastp(leastp&&) = default;

    leastp& operator=(const leastp& other)
    {
        if (this != &other)
        {
            // lifetime of *this ends
            this->~leastp();
            // new object of type
            // leastp created
            new (this) leastp(other);
        }
        return *this;
    }

    leastp& operator=(leastp&& other)
    {
        if (this != &other)
        {
            // lifetime of *this ends
            this->~leastp();
            // new object of type
            // leastp created
            new (this) leastp(other);
        }
        return *this;
    }
    // least privilege
    constexpr size_t size() const
    {
        return v.size();
    }
private:
    std::vector<int>& v;
    const int i;
};

```

```

// least privilege
class leastp final
{
public:
    leastp(std::vector<int>& v,
        : v{v}, i{i} {}

    leastp(const leastp&) = default;
    leastp(leastp&&) = default;
    leastp& operator=(const leastp& other) = default;
    leastp& operator=(leastp&& other) = default;
    // least privilege
    constexpr size_t size() const
    {
        return v.size();
    }
private:
    std::vector<int>& v;
    const int i;
};

```

Motivation

Adding this functionality would make the following enforcement unnecessary.

C.12: Don't make data members `const` or references in a copyable or movable type

[2]

Reason `const` and reference data members are not useful in a copyable or movable type, and make such types difficult to use by making them at least partly uncopyable/unmovable for subtle reasons.

Example; bad

```
class bad {
    const int i;    // bad
    string& s;      // bad
    // ...
};
```

The `const` and `&` data members make this class “only-sort-of-copyable” – copy-constructible but not copy-assignable.

Note If you need a member to point to something, use a pointer (raw or smart, and `gsl::not_null` if it should not be null) instead of a reference.

Enforcement Flag a data member that is `const`, `&`, or `&&` in a type that has any copy or move operation.

Without this functionality, besides being more code, constness has to be enforced via even more code instead of it being enforced by use of the `const` keyword or the constness associated with references. Use of references as class members is intuitive from the standpoint constructor based dependency injection and object proxies which is useful for enforcing least privilege.

Impact on the standard

This feature removes a current limitation of default member functions. Consequently, it is not expected to break anything as this functionality isn't supported.

The proposed changes are relative to the current working draft N5008 ^[1:1].

References

1. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5008.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5008.pdf>)  
2. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#c12-dont-make-data-members-const-or-references-in-a-copyable-or-movable-type>
(<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#c12-dont-make-data-members-const-or-references-in-a-copyable-or-movable-type>) 